```
import java.util.Iterator;

import components.binarytree.BinaryTree;
import components.binarytree.BinaryTree1;
import components.set.Set;
import components.set.SetSecondary;

/**
 * {@code Set} represented as a {@code BinaryTree} (maintained as a binary
 * search tree) of elements with implementations of primary methods.
 *
 * @param <T>
 *            type of {@code Set} elements
 * @mathdefinitions <pre>
 * IS_BST(
 *   tree: binary tree of T
 *  ): boolean satisfies
 *  [tree satisfies the binary search tree properties as described in the
 *   slides with the ordering reported by compareTo for T, including that
 *   it has no duplicate labels]
 * </pre>
 * @convention IS_BST($this.tree)
 * @correspondence this = labels($this.tree)
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He & Meng Zhang)
 *
 */
public class Set3a<T extends Comparable<T>> extends SetSecondary<T> {

    /*
     * Private members --------------------------------------------------------
     */

    /**
     * Elements included in {@code this}.
     */
    private BinaryTree<T> tree;

    /**
     * Returns whether {@code x} is in {@code t}.
     *
     * @param <T>
     *            type of {@code BinaryTree} labels
     * @param t
     *            the {@code BinaryTree} to be searched
     * @param x
     *            the label to be searched for
     * @return true if t contains x, false otherwise
     * @requires IS_BST(t)
     * @ensures isInTree = (x is in labels(t))
     */
    private static <T extends Comparable<T>> boolean isInTree(BinaryTree<T> t,
            T x) {
        assert t != null : "Violation of: t is not null";
        assert x != null : "Violation of: x is not null";

        boolean found = false;
        if (t.size() > 0) {
            BinaryTree<T> left = t.newInstance();
            BinaryTree<T> right = t.newInstance();
            T root = t.disassemble(left, right);
            int compare = x.compareTo(root);
            if (compare == 0) {
                found = true;
            } else if (compare < 0) {
                found = isInTree(left, x);
            } else {
                found = isInTree(right, x);
            }
```

```
            t.assemble(root, left, right);
        }

        return found;
    }

    /**
     * Inserts {@code x} in {@code t}.
     *
     * @param <T>
     *            type of {@code BinaryTree} labels
     * @param t
     *            the {@code BinaryTree} to be searched
     * @param x
     *            the label to be inserted
     * @aliases reference {@code x}
     * @updates t
     * @requires IS_BST(t) and x is not in labels(t)
     * @ensures IS_BST(t) and labels(t) = labels(#t) union {x}
     */
    private static <T extends Comparable<T>> void insertInTree(BinaryTree<T> t,
            T x) {
        assert t != null : "Violation of: t is not null";
        assert x != null : "Violation of: x is not null";

        if (t.size() == 0) {
            t.assemble(x, t.newInstance(), t.newInstance());
        } else {
            BinaryTree<T> left = t.newInstance();
            BinaryTree<T> right = t.newInstance();
            T root = t.disassemble(left, right);
            if (x.compareTo(root) < 0) {
                insertInTree(left, x);
            } else {
                insertInTree(right, x);
            }
            t.assemble(root, left, right);
        }

    }

    /**
     * Removes and returns the smallest (left-most) label in {@code t}.
     *
     * @param <T>
     *            type of {@code BinaryTree} labels
     * @param t
     *            the {@code BinaryTree} from which to remove the label
     * @return the smallest label in the given {@code BinaryTree}
     * @updates t
     * @requires IS_BST(t) and |t| > 0
     * @ensures <pre>
     * IS_BST(t)  and  removeSmallest = [the smallest label in #t]  and
     *  labels(t) = labels(#t) \ {removeSmallest}
     * </pre>
     */
    private static <T> T removeSmallest(BinaryTree<T> t) {
        assert t != null : "Violation of: t is not null";
        assert t.size() > 0 : "Violation of: |t| > 0";

        BinaryTree<T> left = t.newInstance();
        BinaryTree<T> right = t.newInstance();
        T root = t.disassemble(left, right);

        T smallest;
        if (left.size() == 0) {
            smallest = root;
            t.transferFrom(right);
        } else {
```

```
            smallest = removeSmallest(left);
            t.assemble(root, left, right);
        }

        return smallest;
    }

    /**
     * Finds label {@code x} in {@code t}, removes it from {@code t}, and
     * returns it.
     *
     * @param <T>
     *            type of {@code BinaryTree} labels
     * @param t
     *            the {@code BinaryTree} from which to remove label {@code x}
     * @param x
     *            the label to be removed
     * @return the removed label
     * @updates t
     * @requires IS_BST(t) and x is in labels(t)
     * @ensures <pre>
     * IS_BST(t)  and  removeFromTree = x  and
     *  labels(t) = labels(#t) \ {x}
     * </pre>
     */
    private static <T extends Comparable<T>> T removeFromTree(BinaryTree<T> t,
            T x) {
        assert t != null : "Violation of: t is not null";
        assert x != null : "Violation of: x is not null";
        assert t.size() > 0 : "Violation of: x is in labels(t)";

        BinaryTree<T> left = t.newInstance();
        BinaryTree<T> right = t.newInstance();
        T root = t.disassemble(left, right);

        T removed;
        int compare = x.compareTo(root);
        if (compare < 0) {
            removed = removeFromTree(left, x);
            t.assemble(root, left, right);
        } else if (compare > 0) {
            removed = removeFromTree(right, x);
            t.assemble(root, left, right);
        } else {
            removed = root;
            if (left.size() == 0) {
                t.transferFrom(right);
            } else if (right.size() == 0) {
                t.transferFrom(left);
            } else {
                T smallest = removeSmallest(right);
                t.assemble(smallest, left, right);
            }
        }

        return removed;
    }

    /**
     * Creator of initial representation.
     */
    private void createNewRep() {

        this.tree = new BinaryTree1<T>();

    }

    /*
     * Constructors ------------------------------------------------------------
```

```
     */

    /**
     * No-argument constructor.
     */
    public Set3a() {

        this.createNewRep();

    }

    /*
     * Standard methods -----------------------------------------------------
     */

    @SuppressWarnings("unchecked")
    @Override
    public final Set<T> newInstance() {
        try {
            return this.getClass().getConstructor().newInstance();
        } catch (ReflectiveOperationException e) {
            throw new AssertionError(
                    "Cannot construct object of type " + this.getClass());
        }
    }

    @Override
    public final void clear() {
        this.createNewRep();
    }

    @Override
    public final void transferFrom(Set<T> source) {
        assert source != null : "Violation of: source is not null";
        assert source != this : "Violation of: source is not this";
        assert source instanceof Set3a<?> : ""
                + "Violation of: source is of dynamic type Set3a<?>";
        /*
         * This cast cannot fail since the assert above would have stopped
         * execution in that case: source must be of dynamic type Set3a<?>, and
         * the ? must be T or the call would not have compiled.
         */
        Set3a<T> localSource = (Set3a<T>) source;
        this.tree = localSource.tree;
        localSource.createNewRep();
    }

    /*
     * Kernel methods -------------------------------------------------------
     */

    @Override
    public final void add(T x) {
        assert x != null : "Violation of: x is not null";
        assert !this.contains(x) : "Violation of: x is not in this";

        insertInTree(this.tree, x);

    }

    @Override
    public final T remove(T x) {
        assert x != null : "Violation of: x is not null";
        assert this.contains(x) : "Violation of: x is in this";

        return removeFromTree(this.tree, x);
    }

    @Override
```

```
    public final T removeAny() {
        assert this.size() > 0 : "Violation of: this /= empty_set";

        return removeSmallest(this.tree);
    }

    @Override
    public final boolean contains(T x) {
        assert x != null : "Violation of: x is not null";

        return isInTree(this.tree, x);
    }

    @Override
    public final int size() {

        return this.tree.size();
    }

    @Override
    public final Iterator<T> iterator() {
        return this.tree.iterator();
    }

}
```